37-database-intro

37. Pengenalan Database

Level: Beginner **Jam:** 8 (4 minggu × 2 sesi) **Prasyarat:** — **Output:** Database schema + SQL queries untuk aplikasi nyata

Tujuan Pembelajaran

Setelah modul ini, kamu bisa: - Paham konsep database dan bedanya sama spreadsheet - Nulis SQL DML (SELECT, INSERT, UPDATE, DELETE) dengan percaya diri - Mendesain table relationships (1:1, 1:N, N:M) pakai foreign key - Normalisasi database sampai 3NF - Bikin ERD dari requirement aplikasi

Materi

Sesi	Topik	File
1	What is Database — DBMS types, table structure, data types, primary key	01-what-is-database.md
2	SQL Basics — CREATE TABLE, INSERT, SELECT, UPDATE, DELETE, aggregate functions	02-sql-basics.md
3	Relationships — Foreign key, JOIN, table relationships, indexing	03-relationships.md
4	Database Design — Normalization, ERD, naming convention, migration	04-database-design.md

Output Akhir Modul

> **Database Schema untuk Aplikasi Nyata** — desain database lengkap dengan ERD, table definitions, SQL queries, dan migration script untuk aplikasi e-commerce / toko online.

AI Prompt Exercises

Sepanjang modul, latihan pake AI: - “Explain this SQL query step by step” - “Convert this spreadsheet to a normalized database schema” - “Find the normalization issue in this table design” - “Generate 5 test data rows for this table” - “Write a query to find duplicate records in this table”

1.1 What is Database?

Database vs Spreadsheet

Bayangin spreadsheet vs database sebagai tempat nyimpen data:

Aspek	Spreadsheet (Excel/Google Sheets)	Database (PostgreSQL/MySQL)
Struktur	Sel, baris, kolom di 1 file	Table, row, column — banyak table saling terhubung
Relasi	Manual pake VLOOKUP	Pake foreign key & JOIN
Skalabilitas	Ribuan baris mulai lemot	Jutaan baris masih cepat
Multi-user	Rentan conflict	ACID transaction, concurrent safe
Validasi	Manual	Constraint (NOT NULL, UNIQUE, CHECK)
Query	Filter/sort manual	SQL — SELECT, WHERE, JOIN, GROUP BY
Keamanan	Password file	Role-based access control

Kapan pake spreadsheet: Data kecil, ad-hoc, analisis cepat, satu orang. **Kapan pake database:** Data besar, multi-user, butuh konsistensi, aplikasi production.

DBMS Types

DBMS (Database Management System) — software buat manage database.

Relational Database (SQL)

Data disimpan dalam table dengan hubungan (relationship) antar table.

DBMS	Cocok Untuk	Kelebihan
PostgreSQL	Aplikasi kompleks, geolocation, analytics	Fitur lengkap, open source, extensible
MySQL	Web app, CMS, e-commerce	Populer, performa tinggi, banyak hosting support
SQLite	Mobile app, embedded, prototyping	Serverless, file-based, zero config

```sql
-- Semua pake SQL — beda sedikit di syntax
-- PostgreSQL
CREATE TABLE users (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    created_at TIMESTAMP DEFAULT NOW()
);

-- MySQL
CREATE TABLE users (
    id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- SQLite
CREATE TABLE users (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL,
    created_at TEXT DEFAULT (datetime('now'))
);
```

NoSQL Database

Data tidak pakai table kaku — lebih fleksibel buat use case tertentu.

DBMS	Model Data	Cocok Untuk
MongoDB	Document (JSON-like)	Data fleksibel, prototyping cepat
Redis	Key-value	Cache, session store, realtime
Firebase	Document + Realtime	Mobile app, sync cepat

```
// MongoDB — document dalam collection
{
    "_id": ObjectId("..."),
    "name": "Budi",
    "email": "budi@email.com",
    "orders": ["order1", "order2"]  // array reference
}
```

> **Rule of thumb:** Kalau data punya relasi jelas (user punya order, order punya items) → pakai SQL. Kalau data dokumen fleksibel (blog post, config) → NoSQL opsi. Di modul ini kita fokus ke **relational database (PostgreSQL/SQLite)**.

Table Structure

Table di database mirip spreadsheet — tapi lebih strict.

```
erDiagram
    users {
        INT id PK
        VARCHAR name
        VARCHAR email
        INT age
        BOOLEAN is_active
        TIMESTAMP created_at
    }
```

Row vs Column vs Field vs Record

Istilah	Arti	Analogi
Table	Kumpulan data dengan struktur sama	File Excel
Row / Record	Satu baris data	Satu baris di spreadsheet
Column / Field	Satu atribut data	Satu kolom di spreadsheet
Schema	Definisi struktur table	Header kolom

Data Types

Setiap column harus punya tipe data. Ini constraint pertama — data nyangkut.

Tipe Data SQL Lengkap

Tipe Data	Contoh	Ukuran	Kegunaan
`INTEGER` / `INT`	`25`, `-10`, `1000000`	4 bytes	Angka bulat — id, umur, quantity
`BIGINT`	`99999999999`	8 bytes	Angka besar — log ID, counter
`SMALLINT`	`100`, `-5`	2 bytes	Angka kecil — rating 1-5
`VARCHAR(n)`	`'Budi'`, `'Jakarta'`	Variabel	Teks pendek — nama, email, alamat
`TEXT`	Laporan panjang (>255 char)	Variabel	Teks panjang — deskripsi, konten artikel
`BOOLEAN`	`true`, `false`	1 bit	Yes/no — is_active, is_verified
`DATE`	`'2024-01-15'`	4 bytes	Tanggal tanpa waktu — tanggal lahir

Tipe Data	Contoh	Ukuran	Kegunaan
`TIMESTAMP`	`'2024-01-15 10:30:00'`	8 bytes	Tanggal + waktu — created_at, updated_at
`FLOAT` / `REAL`	`3.14`, `0.001`	4 bytes	Angka desimal presisi rendah — rating
`DECIMAL(p,s)` / `NUMERIC`	`150000.50`	Variabel	Uang — harga, gaji (presisi tepat)
`UUID`	`'550e8400-...'`	16 bytes	ID unik global — user_id, order_id
`JSON` / `JSONB`	`'{"key":"val"}'`	Variabel	Data semi-struktur — metadata, config

Penting: - `VARCHAR(255)` vs `TEXT` — VARCHAR bisa di-index penuh, TEXT di-index partial. Buat pencarian exact, VARCHAR lebih cepet. - `FLOAT` vs `DECIMAL` — Jangan pake FLOAT buat uang! FLOAT punya rounding error. `DECIMAL(10,2)` = 10 digit total, 2 desimal. - `TIMESTAMP WITH TIME ZONE` — Selalu pake ini, bukan `TIMESTAMP` biasa. Simpan di UTC, format di aplikasi.

```sql
-- Contoh CREATE TABLE dengan berbagai tipe data
CREATE TABLE products (
    id SERIAL PRIMARY KEY,
    name VARCHAR(200) NOT NULL,
    description TEXT,
    price DECIMAL(10, 2) NOT NULL,
    stock INT DEFAULT 0,
    is_active BOOLEAN DEFAULT true,
    category VARCHAR(50),
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),
    updated_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);
```

Primary Key & Auto Increment

Primary Key (PK): Column (atau kombinasi column) yang unik identify tiap row.

Sifat PK: - `UNIQUE` — tidak boleh duplikat - `NOT NULL` — wajib diisi - Hanya satu PK per table

Auto Increment / Serial — Biarin database generate ID otomatis.

```
-- PostgreSQL — SERIAL (integer)
CREATE TABLE users (
    id SERIAL PRIMARY KEY,  -- otomatis 1, 2, 3, ...
    name VARCHAR(100)
);

-- PostgreSQL modern — IDENTITY (SQL standard)
CREATE TABLE users (
    id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    name VARCHAR(100)
);

-- INSERT — gausah isi id, database handle sendiri
INSERT INTO users (name) VALUES ('Budi'), ('Ani');
-- id: 1, 2 otomatis
```

Natural Key vs Surrogate Key:

Key Type	Contoh	Kelebihan	Kekurangan
Natural Key	`email`, NIK, ISBN	Bermakna secara bisnis	Bisa berubah, panjang
Surrogate Key	`id SERIAL`, `UUID`	Stabil, sederhana	Tidak bermakna

> **Best practice:** Selalu pake surrogate key (`id SERIAL` atau `UUID`) sebagai primary key. Natural key jadi UNIQUE constraint aja.

```
CREATE TABLE users (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  -- UUID
    email VARCHAR(255) UNIQUE NOT NULL,             -- natural key jadi unique
    name VARCHAR(100)
);
```

Tools

Tools buat interaksi sama database:

Tool	Tipe	Kegunaan
pgAdmin	GUI	PostgreSQL admin — query, manage, visual
DBeaver	GUI	Universal — support banyak DBMS
TablePlus	GUI	Modern, fast — macOS/Linux/Windows
Prisma Studio	GUI	Visual data browser buat Prisma ORM
psql	CLI	PostgreSQL native CLI
SQLite Browser	GUI	Explore SQLite file
VS Code SQL Tools	Extension	Query langsung dari editor

Latihan

Latihan 1: Database vs Spreadsheet Bikin tabel perbandingan: kapan lebih baik pake spreadsheet, kapan pake database. Minimal 3 skenario masing-masing.

Latihan 2: Identifikasi DBMS Dari skenario berikut, tentuin DBMS yang paling cocok dan jelaskan kenapa: 1. Aplikasi POS toko kelontong dengan 5 kasir 2. Blog pribadi dengan 10 artikel/bulan 3. Aplikasi chat realtime jutaan user 4. IoT sensor logger — ribuan data per detik 5. Aplikasi mobile catatan offline

Latihan 3: Desain Table & Tipe Data Kamu diminta bikin table `students` buat sistem sekolah. Tentukan column dan tipe data yang tepat untuk: - Nama lengkap - Tanggal lahir - NISN (nomor induk) - Nilai rata-rata (bisa pecahan) - Status aktif/tidak - Alamat (bisa panjang) - Tanggal daftar - Nomor telepon

Tulis dalam bentuk `CREATE TABLE` SQL.

Latihan 4: Primary Key Decision Dari table berikut, tentukan primary key yang tepat (natural vs surrogate) dan jelaskan: 1. Table `books` di perpustakaan — punya ISBN 2. Table `employees` — punya NIK dan email 3. Table `transactions` — transaksi harian ribuan 4. Table `categories` — hanya 10 kategori statis

1.2 SQL Basics

SQL Syntax

SQL (Structured Query Language) — bahasa buat komunikasi sama database.

Kategori SQL

Kategori	Kepanjangan	Perintah
DDL	Data Definition Language	`CREATE`, `ALTER`, `DROP` — define struktur
DML	Data Manipulation Language	`SELECT`, `INSERT`, `UPDATE`, `DELETE` — manipulasi data
DCL	Data Control Language	`GRANT`, `REVOKE` — hak akses
TCL	Transaction Control Language	`COMMIT`, `ROLLBACK`, `BEGIN` — transaksi

Di sesi ini kita fokus ke **DDL** (CREATE TABLE) dan **DML** (INSERT, SELECT, UPDATE, DELETE).

> **Aturan SQL:** - Case-insensitive: `SELECT` == `select` == `Select` (tapi uppercase biar jelas) - String pakai single quote: `'Budi'`, bukan `"Budi"` - Setiap statement diakhiri `;` - Komentar: `-- single line`, `/* multi line */`

CREATE TABLE

Bikin table baru.

```sql
-- Syntax dasar
CREATE TABLE nama_table (
    column1 tipe_data [constraints],
    column2 tipe_data [constraints],
    ...
);

-- Contoh: table products
CREATE TABLE products (
    id SERIAL PRIMARY KEY,
    name VARCHAR(200) NOT NULL,
    price DECIMAL(10, 2) NOT NULL,
    stock INT DEFAULT 0,
    category VARCHAR(100),
    is_active BOOLEAN DEFAULT true,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);
```

Common Constraints

Constraint	Maksud	Contoh
`NOT NULL`	Wajib diisi	`name VARCHAR(100) NOT NULL`
`UNIQUE`	Tidak boleh duplikat	`email VARCHAR(255) UNIQUE`
`PRIMARY KEY`	NOT NULL + UNIQUE	`id SERIAL PRIMARY KEY`
`DEFAULT`	Nilai default	`stock INT DEFAULT 0`
`CHECK`	Validasi kondisi	`price DECIMAL(10,2) CHECK (price > 0)`

```sql
-- CREATE TABLE dengan semua constraint
CREATE TABLE categories (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL UNIQUE,
    slug VARCHAR(100) NOT NULL UNIQUE,
    description TEXT,
    is_active BOOLEAN DEFAULT true,
    sort_order INT DEFAULT 0 CHECK (sort_order >= 0),
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);
```

INSERT

Nambah data ke table.

```sql
-- Syntax
INSERT INTO nama_table (column1, column2, ...)
VALUES (value1, value2, ...);

-- Insert satu baris
INSERT INTO products (name, price, stock, category)
VALUES ('Kopi Arabika 250gr', 45000, 50, 'Kopi');

-- Insert banyak baris
INSERT INTO products (name, price, stock, category) VALUES
    ('Kopi Robusta 250gr', 35000, 30, 'Kopi'),
    ('Teh Hijau 50gr', 25000, 20, 'Teh'),
    ('Gula Aren 500gr', 18000, 100, 'Gula');

-- Insert dengan return (PostgreSQL)
INSERT INTO products (name, price, stock)
VALUES ('Kopi Luwak 100gr', 150000, 5)
RETURNING *;
-- Balikin baris yang baru diinsert beserta id-nya
```

INSERT tanpa specified columns

```sql
-- Kalau sebut column, urutan bebas
INSERT INTO products (name, price, stock) VALUES ('Test', 1000, 5);

-- Kalau tidak sebut column, WAJIB urutan sesuai schema
INSERT INTO products VALUES (DEFAULT, 'Test', 1000, 5, NULL, true, 'now');
-- DEFAULT = ambil default value
-- NULL/true/'now' = isi manual
```

> **Best practice:** Selalu sebut column names. Lebih jelas dan nggak rentan error kalau struktur table berubah.

SELECT

Mengambil data dari table.

```sql
-- Syntax dasar
SELECT column1, column2, ...
FROM nama_table
WHERE kondisi
ORDER BY column [ASC|DESC]
LIMIT jumlah
OFFSET lompat;

-- SELECT semua column
SELECT * FROM products;

-- SELECT spesifik column
SELECT name, price FROM products;

-- SELECT dengan alias
SELECT name AS nama_produk, price AS harga FROM products;

-- SELECT dengan perhitungan
SELECT name, price, price * 1.11 AS harga_ppn FROM products;
```

WHERE — Filter data

```sql
-- Operator perbandingan (=, <, >, <=, >=, <>)
SELECT * FROM products WHERE price < 50000;
SELECT * FROM products WHERE stock >= 10;
SELECT * FROM products WHERE category = 'Kopi';
SELECT * FROM products WHERE category <> 'Teh';  -- <> = tidak sama
```

```sql
-- LIKE — pencarian teks (case-insensitive di PostgreSQL)
SELECT * FROM products WHERE name LIKE '%Kopi%';     -- mengandung "Kopi"
SELECT * FROM products WHERE name LIKE 'Kopi%';      -- diawali "Kopi"
SELECT * FROM products WHERE name LIKE '%250gr';     -- diakhiri "250gr"
-- % = wildcard (0 atau lebih karakter)
-- _ = wildcard 1 karakter

-- IN — filter beberapa nilai
SELECT * FROM products WHERE category IN ('Kopi', 'Teh', 'Gula');

-- BETWEEN — range
SELECT * FROM products WHERE price BETWEEN 20000 AND 50000;
SELECT * FROM products WHERE created_at BETWEEN '2024-01-01' AND '2024-12-31';

-- IS NULL — cek null
SELECT * FROM products WHERE category IS NULL;
SELECT * FROM products WHERE description IS NOT NULL;
```

Logical Operators

```sql
-- AND — semua kondisi harus true
SELECT * FROM products
WHERE category = 'Kopi'
  AND price < 50000
  AND stock > 0;

-- OR — salah satu kondisi true
SELECT * FROM products
WHERE category = 'Kopi'
   OR category = 'Teh';

-- Kombinasi AND + OR — pake parentheses biar jelas
SELECT * FROM products
WHERE (category = 'Kopi' OR category = 'Teh')
  AND price < 50000;

-- NOT — kebalikan
SELECT * FROM products
WHERE NOT category = 'Teh';
```

ORDER BY — Sorting

```sql
-- ASC = ascending (A-Z, kecil-besar) — default
-- DESC = descending (Z-A, besar-kecil)
```

```sql
SELECT name, price FROM products ORDER BY price ASC;
SELECT name, price FROM products ORDER BY price DESC;
SELECT * FROM products ORDER BY category ASC, price DESC;
-- Sort category A-Z, lalu dalam tiap category sort price besar-kecil
```

LIMIT & OFFSET — Pagination

```sql
-- LIMIT: ambil N baris
SELECT * FROM products LIMIT 5;

-- LIMIT + OFFSET: loncati N baris dulu
SELECT * FROM products ORDER BY id LIMIT 10 OFFSET 0;   -- halaman 1
SELECT * FROM products ORDER BY id LIMIT 10 OFFSET 10;  -- halaman 2
SELECT * FROM products ORDER BY id LIMIT 10 OFFSET 20;  -- halaman 3
```

UPDATE

Edit data existing.

```sql
-- Syntax
UPDATE nama_table
SET column1 = value1, column2 = value2, ...
WHERE kondisi;

-- Update satu baris (pasti pake WHERE id)
UPDATE products
SET price = 48000, updated_at = NOW()
WHERE id = 1;

-- Update banyak baris
UPDATE products
SET price = price * 1.1
WHERE category = 'Kopi';
-- Naikin 10% harga semua produk kopi

-- Update dengan RETURNING (PostgreSQL)
UPDATE products
SET stock = stock + 10
WHERE category = 'Teh'
RETURNING id, name, stock;
```

> ⚠ **PERINGATAN:** UPDATE tanpa WHERE = update SEMUA baris. Selalu double check WHERE clause sebelum execute!

DELETE

Hapus data.

```
-- Syntax
DELETE FROM nama_table
WHERE kondisi;

-- Hapus satu baris
DELETE FROM products WHERE id = 5;

-- Hapus dengan kondisi
DELETE FROM products
WHERE stock = 0 AND is_active = false;

-- Hapus dengan RETURNING (PostgreSQL)
DELETE FROM products
WHERE category IS NULL
RETURNING id, name;
```

> ⚠ **PERINGATAN:** DELETE tanpa WHERE = hapus SEMUA data. Sama berbahayanya sama UPDATE tanpa WHERE.

Aggregate Functions

Fungsi yang operasi di sekelompok baris dan balikin satu nilai.

Function	Kegunaan	Contoh
`COUNT()`	Hitung jumlah baris	`COUNT(*)` = total baris, `COUNT(column)` = non-null
`SUM()`	Total nilai	`SUM(price)`
`AVG()`	Rata-rata	`AVG(price)`
`MIN()`	Nilai minimum	`MIN(price)`
`MAX()`	Nilai maksimum	`MAX(price)`

```
-- COUNT
SELECT COUNT(*) AS total_produk FROM products;
SELECT COUNT(DISTINCT category) AS total_kategori FROM products;
-- DISTINCT = hitung unik

-- SUM
SELECT SUM(stock) AS total_stok FROM products;
```

```sql
SELECT SUM(price * stock) AS total_nilai_inventaris FROM products;

-- AVG
SELECT AVG(price) AS harga_rata_rata FROM products;

-- MIN & MAX
SELECT MIN(price) AS termurah, MAX(price) AS termahal FROM products;

-- Kombinasi
SELECT
    COUNT(*) AS total_produk,
    AVG(price)::DECIMAL(10,2) AS rata_harga,
    MIN(price) AS harga_min,
    MAX(price) AS harga_max,
    SUM(stock) AS total_stok
FROM products
WHERE is_active = true;
```

GROUP BY & HAVING

GROUP BY — kelompokkan baris berdasarkan nilai yang sama, lalu apply aggregate.

```sql
-- Syntax
SELECT column, aggregate_function(column)
FROM nama_table
WHERE kondisi
GROUP BY column
HAVING kondisi_group
ORDER BY column;

-- Jumlah produk per kategori
SELECT
    category,
    COUNT(*) AS jumlah_produk,
    AVG(price)::DECIMAL(10,2) AS rata_harga
FROM products
GROUP BY category;

-- Total stok per kategori
SELECT
    category,
    SUM(stock) AS total_stok
FROM products
```

```sql
GROUP BY category
ORDER BY total_stok DESC;

-- Rata-rata harga per kategori — hanya kategori dengan > 1 produk
SELECT
    category,
    COUNT(*) AS jumlah_produk,
    AVG(price)::DECIMAL(10,2) AS rata_harga
FROM products
GROUP BY category
HAVING COUNT(*) > 1;

-- Filter dulu WHERE, baru GROUP BY, baru HAVING
SELECT
    category,
    COUNT(*) AS active_products,
    AVG(price)::DECIMAL(10,2) AS rata_harga
FROM products
WHERE is_active = true            -- filter baris sebelum grouping
GROUP BY category
HAVING COUNT(*) >= 2              -- filter hasil grouping
ORDER BY active_products DESC;
```

Urutan eksekusi SQL (logis, bukan urutan nulis):

1. `FROM` — ambil data dari table
2. `WHERE` — filter baris
3. `GROUP BY` — kelompokkan
4. `HAVING` — filter kelompok
5. `SELECT` — pilih column + aggregate
6. `ORDER BY` — sorting
7. `LIMIT / OFFSET` — pagination

Contoh Query Lengkap

```sql
-- Setup table
CREATE TABLE products (
    id SERIAL PRIMARY KEY,
    name VARCHAR(200) NOT NULL,
    price DECIMAL(10, 2) NOT NULL,
    stock INT DEFAULT 0,
    category VARCHAR(100),
    is_active BOOLEAN DEFAULT true,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);
```

```sql
-- Seed data
INSERT INTO products (name, price, stock, category, is_active) VALUES
    ('Kopi Arabika 250gr', 45000, 50, 'Kopi', true),
    ('Kopi Robusta 250gr', 35000, 30, 'Kopi', true),
    ('Teh Hijau 50gr', 25000, 20, 'Teh', true),
    ('Teh Melati 40gr', 22000, 0, 'Teh', true),
    ('Gula Aren 500gr', 18000, 100, 'Gula', true),
    ('Gula Pasir 1kg', 15500, 0, 'Gula', false),
    ('Kopi Luwak 100gr', 150000, 5, 'Kopi', true),
    ('Teh Tarik 3in1 10sachet', 12000, 40, 'Teh', true);

-- 1. Produk aktif dengan harga di atas rata-rata
SELECT name, price, category
FROM products
WHERE is_active = true
  AND price > (SELECT AVG(price) FROM products);

-- 2. Kategori dengan total stok > 30
SELECT
    category,
    SUM(stock) AS total_stok,
    COUNT(*) AS jumlah_produk
FROM products
WHERE is_active = true
GROUP BY category
HAVING SUM(stock) > 30
ORDER BY total_stok DESC;

-- 3. Produk terlaris (stok abis)
SELECT name, category, price
FROM products
WHERE stock = 0 AND is_active = true;

-- 4. Cari produk berdasarkan kata kunci
SELECT *
FROM products
WHERE name ILIKE '%kopi%'  -- ILIKE = case-insensitive LIKE (PostgreSQL)
   OR name ILIKE '%teh%';
```

Latihan

Latihan 1: Bikin Table Product Buat table `products` dengan kolom:
- `id` — primary key auto increment - `name` — varchar 200, not null -

`price` — decimal 10,2, not null - `stock` — integer, default 0 - `category` — varchar 100 - `is_active` — boolean, default true - `description` — text, nullable - `created_at` — timestamp dengan timezone, default now

Latihan 2: Insert Data Insert 10 produk fiktif ke table products, dengan variasi: - 3 kategori berbeda - Harga dari 5000 sampai 200000 - Stok ada yang 0, ada yang banyak - 2 produk tidak aktif

Latihan 3: Query dengan Kondisi Tulis query untuk: 1. Tampilkan semua produk aktif, urut termurah ke termahal 2. Tampilkan produk yang stoknya habis (0) tapi masih aktif 3. Tampilkan produk dengan harga antara 20000 dan 100000, dari kategori Kopi atau Teh 4. Cari produk yang namanya mengandung kata "Gula" 5. Tampilkan 5 produk termahal yang aktif 6. Tampilkan produk dari halaman 2 (pagination 5 per halaman)

Latihan 4: Aggregation Report Tulis query untuk: 1. Total stok semua produk aktif 2. Rata-rata harga per kategori (tampilkan kategori, rata-rata, jumlah produk) 3. Kategori yang punya total nilai inventaris (price × stock) di atas 1 juta 4. Produk termahal, termurah, dan rata-rata harga semua produk aktif

1.3 Relationships

Foreign Key Concept

Foreign Key (FK): Column di satu table yang refer ke Primary Key di table lain. Ini yang bikin relational database powerful.

```
erDiagram
    users ||--o{ orders : "memiliki"
    users {
        int id PK
        varchar name
        varchar email
    }
    orders {
        int id PK
        int user_id FK
        decimal total
        timestamp created_at
    }
```

```
-- Table users (parent)
CREATE TABLE users (
    id SERIAL PRIMARY KEY,
```

```
    name VARCHAR(100) NOT NULL,
    email VARCHAR(255) UNIQUE NOT NULL
);

-- Table orders (child) dengan foreign key
CREATE TABLE orders (
    id SERIAL PRIMARY KEY,
    user_id INT NOT NULL,
    total DECIMAL(10, 2) NOT NULL DEFAULT 0,
    status VARCHAR(20) DEFAULT 'pending',
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW(),

    -- Foreign key constraint
    CONSTRAINT fk_orders_user
        FOREIGN KEY (user_id)
        REFERENCES users(id)
        ON DELETE CASCADE
);
```

FK Constraints Actions

Yang terjadi kalau data di parent di-delete atau di-update:

Action	Maksud
`ON DELETE CASCADE`	Delete child otomatis kalau parent di-delete
`ON DELETE SET NULL`	Set FK jadi NULL kalau parent di-delete
`ON DELETE RESTRICT`	Cegah delete parent kalau masih ada child
`ON DELETE NO ACTION`	Sama kaya RESTRICT, deferred di beberapa DB
`ON UPDATE CASCADE`	Update FK otomatis kalau PK parent berubah

```
-- Paling umum dipake
CONSTRAINT fk_orders_user
    FOREIGN KEY (user_id)
    REFERENCES users(id)
    ON DELETE CASCADE;

-- Kalo user dihapus, ordernya ikut kehapus

-- Alternatif: user dihapus, order tetap ada tapi user_id jadi NULL
CONSTRAINT fk_orders_user
    FOREIGN KEY (user_id)
    REFERENCES users(id)
    ON DELETE SET NULL;
```

Pilih yang mana? `CASCADE` — kalau data child nggak berguna tanpa parent (kaya order tanpa user). `SET NULL`

— kalau data child tetap berguna (kaya log activity, histori).
RESTRICT — kalau mau jaga integritas ketat.

Table Relationships

1:1 (One-to-One)

Satu row di Table A berhubungan dengan **tepat satu** row di Table B.

Contoh: User ↔ Profile

```
erDiagram
    users ||--o| profiles : "memiliki"
    users {
        int id PK
        varchar name
    }
    profiles {
        int id PK
        int user_id FK
        varchar bio
        date birth_date
        varchar avatar_url
    }
```

```sql
CREATE TABLE profiles (
    id SERIAL PRIMARY KEY,
    user_id INT UNIQUE NOT NULL,            -- UNIQUE = 1:1
    bio TEXT,
    birth_date DATE,
    avatar_url VARCHAR(500),

    CONSTRAINT fk_profile_user
        FOREIGN KEY (user_id)
        REFERENCES users(id)
        ON DELETE CASCADE
);
```

UNIQUE di FK yang bikin 1:1. Tanpa UNIQUE, user bisa punya banyak profile (jadi 1:N).

1:N (One-to-Many)

Satu row di Table A berhubungan dengan **banyak** row di Table B.

Contoh: User → Orders, Category → Products

```
erDiagram
    categories ||--o{ products : "memiliki"
    categories {
        int id PK
        varchar name
    }
    products {
        int id PK
        int category_id FK
        varchar name
        decimal price
    }
```

```sql
CREATE TABLE categories (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL
);

CREATE TABLE products (
    id SERIAL PRIMARY KEY,
    category_id INT NOT NULL,
    name VARCHAR(200) NOT NULL,
    price DECIMAL(10, 2) NOT NULL,

    CONSTRAINT fk_product_category
        FOREIGN KEY (category_id)
        REFERENCES categories(id)
        ON DELETE RESTRICT
);

-- 1 kategori bisa punya banyak produk
-- 1 produk hanya punya 1 kategori
```

N:M (Many-to-Many)

Satu row di Table A berhubungan dengan **banyak** row di Table B, dan sebaliknya.

Contoh: Students ↔ Courses, Products ↔ Tags

```
erDiagram
    students ||--o{ enrollments : "mengikuti"
    courses ||--o{ enrollments : "diikuti"
    enrollments {
        int student_id FK
        int course_id FK
        date enrolled_date
```

```
        varchar grade
    }
    students {
        int id PK
        varchar name
    }
    courses {
        int id PK
        varchar title
    }
```

N:M butuh **junction table** (table penghubung) di tengah.

```sql
CREATE TABLE students (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL
);

CREATE TABLE courses (
    id SERIAL PRIMARY KEY,
    title VARCHAR(200) NOT NULL
);

-- Junction table
CREATE TABLE enrollments (
    student_id INT NOT NULL,
    course_id INT NOT NULL,
    enrolled_date DATE DEFAULT CURRENT_DATE,
    grade VARCHAR(2),

    -- Composite primary key — 2 column jadi 1 PK
    PRIMARY KEY (student_id, course_id),

    CONSTRAINT fk_enrollment_student
        FOREIGN KEY (student_id)
        REFERENCES students(id)
        ON DELETE CASCADE,

    CONSTRAINT fk_enrollment_course
        FOREIGN KEY (course_id)
        REFERENCES courses(id)
        ON DELETE CASCADE
);

-- 1 student bisa ambil banyak course
-- 1 course bisa diambil banyak student
```

Ringkasan Relationship

Tipe	FK di table mana?	Butuh extra?	Contoh
1:1	Salah satu table	UNIQUE constraint di FK	user → profile
1:N	Table child (many side)	—	category → products
N:M	Junction table	Table ketiga	students ↔ courses

JOIN Types

JOIN = menggabungkan data dari 2+ table berdasarkan hubungan.

INNER JOIN

Hanya baris yang **punya pasangan** di kedua table.

```
graph TD
    subgraph A[Table A]
        A1[a1]
        A2[a2]
        A3[a3]
    end
    subgraph B[Table B]
        B1[b1]
        B2[b2]
        B3[b3]
    end
    A1 --- B1
    A2 --- B2
    style A1 fill:#6abf69
    style A2 fill:#6abf69
    style A3 fill:#ddd
    style B1 fill:#6abf69
    style B2 fill:#6abf69
    style B3 fill:#ddd
```

```sql
-- INNER JOIN
SELECT * FROM table_a
INNER JOIN table_b ON table_a.id = table_b.fk_id;

-- Contoh: produk + kategori
SELECT
    p.name AS produk,
    c.name AS kategori,
```

```
    p.price
FROM products p
INNER JOIN categories c ON p.category_id = c.id;
-- Hanya produk yang punya kategori — produk tanpa kategori tidak muncul
```

LEFT JOIN

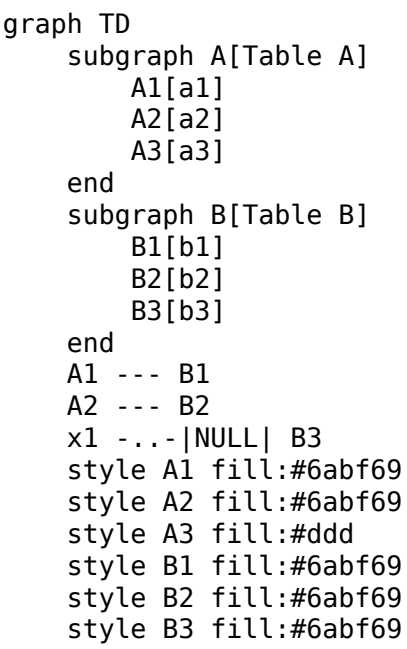
Semua baris dari Table A, plus data Table B kalau ada (kalau nggak ada → NULL).

```
graph TD
    subgraph A[Table A]
        A1[a1]
        A2[a2]
        A3[a3]
    end
    subgraph B[Table B]
        B1[b1]
        B2[b2]
        B3[b3]
    end
    A1 --- B1
    A2 --- B2
    A3 -..-|NULL| x1
    style A1 fill:#6abf69
    style A2 fill:#6abf69
    style A3 fill:#6abf69
    style B1 fill:#6abf69
    style B2 fill:#6abf69
    style B3 fill:#ddd
```

```
-- LEFT JOIN
SELECT * FROM table_a
LEFT JOIN table_b ON table_a.id = table_b.fk_id;

-- Contoh: semua kategori, meskipun tidak punya produk
SELECT
    c.name AS kategori,
    COUNT(p.id) AS jumlah_produk
FROM categories c
LEFT JOIN products p ON c.id = p.category_id
GROUP BY c.id, c.name
ORDER BY jumlah_produk DESC;
```

RIGHT JOIN

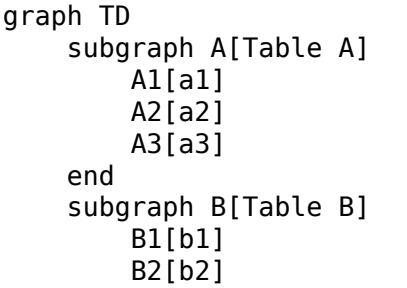
Kebalikan LEFT JOIN — semua baris dari Table B, plus data Table A kalau ada.




```sql
-- RIGHT JOIN — jarang dipake, biasanya dibalik pake LEFT saja
SELECT * FROM table_a
RIGHT JOIN table_b ON table_a.id = table_b.fk_id;

-- Sama aja kaya:
SELECT * FROM table_b
LEFT JOIN table_a ON table_b.id = table_a.fk_id;
```

FULL JOIN

Semua baris dari kedua table — kalau nggak ada pasangan → NULL.



```
        B3[b3]
    end
    A1 --- B1
    A2 --- B2
    A3 -..-|NULL| x1
    x2 -..-|NULL| B3
    style A1 fill:#6abf69
    style A2 fill:#6abf69
    style A3 fill:#6abf69
    style B1 fill:#6abf69
    style B2 fill:#6abf69
    style B3 fill:#6abf69
```

```sql
-- FULL JOIN — semua muncul, yang tidak cocok jadi NULL
SELECT * FROM table_a
FULL JOIN table_b ON table_a.id = table_b.fk_id;
```

Venn Diagram JOIN

```
INNER JOIN:      A ∩ B          [●-------●]
LEFT JOIN:       A ∪ (A∩B)      [●-------●] + A saja
RIGHT JOIN:      B ∪ (A∩B)      [●-------●] + B saja
FULL JOIN:      A ∪ B        [●-------●] + A saja + B saja
```

Contoh JOIN Lengkap

```sql
-- Setup data
CREATE TABLE users (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100)
);
CREATE TABLE orders (
    id SERIAL PRIMARY KEY,
    user_id INT,
    total DECIMAL(10,2),
    FOREIGN KEY (user_id) REFERENCES users(id)
);

INSERT INTO users (name) VALUES ('Budi'), ('Ani'), ('Citra');
INSERT INTO orders (user_id, total) VALUES
    (1, 150000), (1, 75000), (2, 200000);

-- INNER JOIN — user yang punya order
SELECT u.name, o.total, o.id AS order_id
FROM users u
INNER JOIN orders o ON u.id = o.user_id;
```

```sql
-- Hasil: Budi (2 order), Ani (1 order)
-- Citra tidak muncul (tidak punya order)

-- LEFT JOIN — semua user + order mereka
SELECT u.name, o.total
FROM users u
LEFT JOIN orders o ON u.id = o.user_id;
-- Hasil: Budi (150000, 75000), Ani (200000), Citra (NULL)
-- Citra muncul dengan total NULL

-- LEFT JOIN — user yang tidak punya order
SELECT u.name
FROM users u
LEFT JOIN orders o ON u.id = o.user_id
WHERE o.id IS NULL;
-- Hasil: Citra
```

Indexing Basics

Index = struktur data (B-Tree) yang bikin pencarian cepet.

Kenapa Index?

```sql
-- Tanpa index — database scan SEMUA baris (full table scan)
SELECT * FROM products WHERE name = 'Kopi Arabika 250gr';
-- Kalau 1 juta baris, dicek satu-satu → lambat

-- Dengan index — langsung lompat ke baris yang cocok
CREATE INDEX idx_products_name ON products(name);
SELECT * FROM products WHERE name = 'Kopi Arabika 250gr';
-- Langsung ketemu → cepet
```

Kapan pake index?

Column	Index?	Alasan
`id` (PK)	✅ Otomatis	Primary key always indexed
`email`	✅	Sering dipake login/WHERE
`name`	✅	Sering di-search
`created_at`	✅	Sering di-ORDER BY atau BETWEEN
`description` (TEXT)	❌	Jarang di-filter exact match
`is_active` (BOOLEAN)	❌	Hanya 2 nilai — selectivity rendah

Jenis Index

```sql
-- Single column index
CREATE INDEX idx_users_email ON users(email);

-- Composite index (multi column) — urutan penting!
CREATE INDEX idx_users_status_created
ON users(is_active, created_at DESC);
-- Berguna buat: WHERE is_active = true ORDER BY created_at DESC

-- Unique index (sama kayak UNIQUE constraint)
CREATE UNIQUE INDEX idx_users_email_unique ON users(email);

-- Partial index — cuma index baris tertentu
CREATE INDEX idx_active_products ON products(is_active)
WHERE is_active = true;
-- Hemat space, cepet kalau sering filter aktif
```

Trade-off Index

Kelebihan	Kekurangan
SELECT lebih cepet	INSERT/UPDATE/DELETE lebih lambat (harus update index juga)
ORDER BY lebih cepet	Pake space disk
JOIN lebih cepet	Index salah bikin database bingung

> **Rule of thumb:** Index column yang sering dipake di WHERE, JOIN, ORDER BY. Jangan index SEMUA column. Mulai dari index di PK (otomatis) + FK + column yang sering difilter.

Migration Concept

Migration = version control buat database schema.

Kenapa Migration?

```
Tim development — 3 orang, 3 database lokal, 1 production

Tanpa migration:
- Budi nambah column "phone" di table users
```

- Ani gak tau — codenya error karena column belum ada
- Production di-update manual — beda sama lokal
- Siapa yang terakhir update? Nggak jelas

Dengan migration:
- migration_001_add_phone_to_users.sql — semua jalanin
- Urutan jelas, versioning jelas
- Bisa rollback kalau error

Contoh Migration

```sql
-- File: 001_create_users.sql
-- Up
CREATE TABLE users (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(255) UNIQUE NOT NULL,
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

-- Down (rollback)
DROP TABLE users;

-- File: 002_add_phone_to_users.sql
-- Up
ALTER TABLE users ADD COLUMN phone VARCHAR(20);

-- Down
ALTER TABLE users DROP COLUMN phone;

-- File: 003_create_orders.sql
-- Up
CREATE TABLE orders (
    id SERIAL PRIMARY KEY,
    user_id INT NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    total DECIMAL(10, 2) NOT NULL,
    status VARCHAR(20) DEFAULT 'pending',
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);

CREATE INDEX idx_orders_user_status ON orders(user_id, status);

-- Down
DROP TABLE orders;
```

Tools Migration

Tool	Bahasa	Database
knex.js	Node.js	Multi DB
Prisma Migrate	Node.js	Multi DB (auto-generate)
Alembic	Python	SQLAlchemy
Flyway	Java	Multi DB
golang-migrate	Go	Multi DB
Manual SQL	—	Universal

Latihan

Latihan 1: Desain Users + Orders Buat table `users` dan `orders` dengan relasi 1:N: - Users: id, name, email, phone, created_at - Orders: id, user_id (FK), total, status (pending/paid/shipped/delivered/cancelled), shipping_address, created_at - Tambah index di `user_id` dan `status` - Tulis constraint ON DELETE yang tepat

Latihan 2: Query dengan JOIN Dari table users dan orders di atas, tulis query untuk: 1. Mendapatkan semua order milik user dengan nama "Budi", termasuk detail user 2. Menampilkan semua user beserta total belanja mereka (user yang belum order tetap muncul) 3. Menampilkan user yang belum pernah order 4. Total pendapatan (SUM total) per status order 5. Rata-rata total order per user (hanya user yang pernah order)

Latihan 3: Categories N:M dengan Products Buat sistem kategori yang memungkinkan satu produk punya banyak kategori:

- `products` — id, name, price, stock
- `categories` — id, name, slug
- `product_categories` — junction table

Tulis query untuk: 1. Menambahkan produk "Kopi Arabika" ke kategori "Kopi" dan "Premium" 2. Mendapatkan semua kategori dari satu produk 3. Mendapatkan semua produk dalam satu kategori 4. Produk yang tidak punya kategori sama sekali

Latihan 4: Add Indexing Dari semua table yang udah dibuat, identifikasi 3 query yang paling sering dipake. Tambah index yang sesuai. Tuliskan: - Query yang mau dipercepat - Index yang dibuat (CREATE INDEX) - Kenapa index itu membantu

1.4 Database Design

Normalization

Normalization = proses menghilangkan data duplikat dan menjaga integritas data dengan memecah table jadi table-table kecil yang terstruktur.

Tujuan: - □ Hindari data redundancy (data sama di banyak tempat) - □ Hindari anomaly (INSERT/UPDATE/DELETE error karena duplikasi) - □ Jaga konsistensi data - □ Memudahkan maintenance

Tiga bentuk normal paling umum: **1NF, 2NF, 3NF**.

1NF — First Normal Form

Aturan: 1. Setiap column punya satu nilai (atomic) — jangan array/string multi-value 2. Setiap baris harus unik (pake PK) 3. Semua column di satu table harus same type

```
-- □ BEFORE — MELANGGAR 1NF
-- Column "subjects" isi banyak nilai dalam satu cell
CREATE TABLE students_1nf_bad (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100),
    subjects VARCHAR(200)  -- 'Math,Science,English' — SATU CELL BANYAK NILAI
);

-- □ Atau pake array
CREATE TABLE students_1nf_bad2 (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100),
    subjects TEXT[]  -- array — tetap melanggar 1NF
);
```

```
-- □ AFTER — 1NF compliant
-- Satu baris per subject per student
CREATE TABLE students_1nf_good (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100),
    subject VARCHAR(100)  -- 'Math' — satu cell SATU nilai
);

-- Data:
-- 1, 'Budi', 'Math'
-- 1, 'Budi', 'Science'
```

```
-- 1, 'Budi', 'English'
-- 2, 'Ani', 'Math'
-- 2, 'Ani', 'Art'
```

2NF — Second Normal Form

Prasyarat: Sudah 1NF. **Aturan:** Semua non-key column harus **fully functionally dependent** pada seluruh PK (bukan sebagian).

Ini cuma masalah kalau PK-nya **composite** (2+ column jadi PK).

```
-- ❌ BEFORE — MELANGGAR 2NF
-- PK composite: (student_id, course_id)
-- "student_name" cuma tergantung student_id, bukan (student_id + course_id)
-- "course_title" cuma tergantung course_id, bukan (student_id + course_id)

CREATE TABLE enrollments_2nf_bad (
    student_id INT,
    course_id INT,
    student_name VARCHAR(100),    -- ❌ partial dependency ke student_id saja
    course_title VARCHAR(200),    -- ❌ partial dependency ke course_id saja
    grade VARCHAR(2),             -- ✅ full dependency ke (student_id, course_id)

    PRIMARY KEY (student_id, course_id)
);

-- Data duplikat:
-- 1, 101, 'Budi', 'Math 101', 'A'
-- 1, 102, 'Budi', 'Science 101', 'B'
-- Nama 'Budi' duplikat di 2 baris

-- ✅ AFTER — 2NF compliant
-- Pisah ke 3 table

CREATE TABLE students_2nf (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL
);

CREATE TABLE courses_2nf (
    id SERIAL PRIMARY KEY,
    title VARCHAR(200) NOT NULL
);

CREATE TABLE enrollments_2nf (
    student_id INT REFERENCES students_2nf(id),
    course_id INT REFERENCES courses_2nf(id),
```

```
    grade VARCHAR(2),

    PRIMARY KEY (student_id, course_id)
);

-- Sekarang "Budi" cuma di satu tempat — students_2nf
-- "Math 101" cuma di satu tempat — courses_2nf
```

3NF — Third Normal Form

Prasyarat: Sudah 2NF. **Aturan:** Tidak ada **transitive dependency** — column non-key tidak boleh tergantung sama column non-key lain.

```
-- ❌ BEFORE — MELANGGAR 3NF
-- "category_name" tergantung "category_id" — tapi category_id bukan PK
-- Transitive: order_id → category_id → category_name

CREATE TABLE products_3nf_bad (
    id SERIAL PRIMARY KEY,
    name VARCHAR(200),
    category_id INT,
    category_name VARCHAR(100),  -- ❌ transitive dependency via category_id
    price DECIMAL(10,2)
);

-- Kalau kategori ganti nama, update di semua produk — repot & rawan inconsisten

-- ✅ AFTER — 3NF compliant

CREATE TABLE categories_3nf (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL
);

CREATE TABLE products_3nf (
    id SERIAL PRIMARY KEY,
    name VARCHAR(200) NOT NULL,
    category_id INT REFERENCES categories_3nf(id),
    price DECIMAL(10, 2) NOT NULL
);

-- Ganti nama kategori cukup di satu baris categories_3nf
-- Produk otomatis pake nama kategori terbaru via JOIN
```

Contoh Normalisasi Lengkap

Skenario: Sistem order sederhana.

Data awal (unnormalized):

OrderID	Customer	Address	Product	Qty	Price	Total
1	Budi	Jl. Merdeka 1, Jakarta	Kopi 250gr, Teh 50gr	2, 1	45000, 25000	115000
2	Ani	Jl. Sudirman 5, Bandung	Gula 1kg	3	15500	46500

Masalah: - `Customer + Address` duplikat kalau order lagi - `Product` multi-value (1 cell) - Duplikasi data

1NF — atomic values:

OrderID	Customer	Address	Product	Qty	Price	Total
1	Budi	Jl. Merdeka 1, Jakarta	Kopi 250gr	2	45000	115000
1	Budi	Jl. Merdeka 1, Jakarta	Teh 50gr	1	25000	115000
2	Ani	Jl. Sudirman 5, Bandung	Gula 1kg	3	15500	46500

Masalah baru: `Customer`, `Address`, `Total` duplikat di baris 1 & 2.

2NF — pisah dependent column:

Orders:

OrderID	Customer	Address	Total
1	Budi	Jl. Merdeka 1, Jakarta	115000
2	Ani	Jl. Sudirman 5, Bandung	46500

Order Items:

OrderID	Product	Qty	Price
1	Kopi 250gr	2	45000
1	Teh 50gr	1	25000
2	Gula 1kg	3	15500

Masalah: `Customer` dan `Address` masih di Orders — transitive dependency.

3NF — no transitive dependency:

Customers:

CustomerID	Name	Address
1	Budi	Jl. Merdeka 1, Jakarta
2	Ani	Jl. Sudirman 5, Bandung

Orders:

OrderID	CustomerID	Total
1	1	115000
2	2	46500

Order Items:

OrderID	Product	Qty	Price
1	Kopi 250gr	2	45000
1	Teh 50gr	1	25000
2	Gula 1kg	3	15500

Products:

ProductID	Name	Price
1	Kopi 250gr	45000
2	Teh 50gr	25000
3	Gula 1kg	15500

Order Items (final v2):

OrderID	ProductID	Qty
1	1	2
1	2	1
2	3	3

Hasil: Zero duplication, semua data konsisten, ganti harga produk cukup di satu tempat.

Denormalization Trade-offs

Kadang normalisasi **berlebihan** — query jadi terlalu banyak JOIN yang bikin lambat.

Denormalization = sengaja taruh data duplikat biar query cepet.

Kapan Denormalize?

Skenario	Normalized	Denormalized
Dashboard analytics	JOIN 5 table	1 table, siap display
Search dengan banyak filter	JOIN 7 table	Flat table dengan index
Read-heavy app	JOIN tiap request	Cache di column ekstra

Contoh Denormalization

```sql
-- Normalized (3NF)
SELECT
    o.id AS order_id,
    c.name AS customer_name,
    SUM(oi.qty * p.price) AS total
FROM orders o
JOIN customers c ON o.customer_id = c.id
JOIN order_items oi ON o.id = oi.order_id
JOIN products p ON oi.product_id = p.id
WHERE o.id = 1
GROUP BY o.id, c.name;

-- Denormalized — total sudah disimpan di orders
SELECT o.id, c.name, o.total
FROM orders o
JOIN customers c ON o.customer_id = c.id
WHERE o.id = 1;
```

Trade-off:

Normalized	Denormalized
✅ Data konsisten	❌ Bisa inconsistent (kalau lupa update)
✅ Update di 1 tempat	❌ Update di banyak tempat
❌ Query banyak JOIN	✅ Query cepet
❌ Read lambat (banyak JOIN)	✅ Read cepet

> **Rule of thumb:** Normalize sampai 3NF dulu. Denormalize **hanya** kalau ada masalah performa yang terukur. Jangan denormalize dari awal.

Naming Conventions

Konsistensi nama = readability + maintainability.

Table Names

```
-- ✅ GOOD — snake_case, plural
CREATE TABLE users (...);
CREATE TABLE order_items (...);
CREATE TABLE product_categories (...);

-- ❌ BAD
CREATE TABLE User (...);        -- PascalCase + singular
CREATE TABLE tbl_user (...);    -- Prefix 'tbl_'
CREATE TABLE userData (...);    -- camelCase
CREATE TABLE user_list (...);   -- Plural vs singular? Pilih salah satu
```

Column Names

```
-- ✅ GOOD — snake_case, singular
id, name, email, created_at, is_active, category_id

-- ❌ BAD
UserName, userdata, UserName, usersName
```

Convention Guide

Elemen	Convention	Contoh
Table name	snake_case, plural	users, order_items, product_categories
Column name	snake_case, singular	first_name, created_at
Primary key	id	id
Foreign key	singular_table_id	user_id, category_id
Boolean	is_ or has_ prefix	is_active, is_verified, has_discount

Elemen	Convention	Contoh
Timestamp	_at suffix	created_at, updated_at, deleted_at
Date	_date suffix	birth_date, order_date
Junction table	table1_table2	product_categories, user_roles

ERD Basics

ERD (Entity Relationship Diagram) — diagram yang menunjukkan entity (table) dan hubungannya.

Notasi Crow's Foot

```
o|----o|  one
||----||  many
o|----||  zero or many (optional)
 ||----o|  one and only one

Relasi:
  Users 1———< Orders     (1:N — user punya banyak order)
  Users 1———1 Profiles   (1:1 — user punya 1 profile)
  Students >———< Courses  (N:M — via junction)
```

Contoh ERD E-commerce

```
erDiagram
    users ||--o{ orders : "memesan"
    users ||--o| profiles : "memiliki"
    orders ||--|{ order_items : "berisi"
    products ||--o{ order_items : "termasuk"
    products }o--|| categories : "dalam"
    products }o--o{ tags : "ditandai"
    product_tags }o--|| tags : "adalah"

    users {
        int id PK
        varchar name
        varchar email
        varchar password_hash
```

```
    boolean is_active
    timestamp created_at
}

profiles {
    int id PK
    int user_id FK
    text bio
    date birth_date
    varchar avatar_url
    varchar phone
}

orders {
    int id PK
    int user_id FK
    decimal total
    varchar status
    text shipping_address
    timestamp created_at
}

order_items {
    int order_id PK,FK
    int product_id PK,FK
    int quantity
    decimal price_at_purchase
}

products {
    int id PK
    varchar name
    text description
    decimal price
    int stock
    int category_id FK
    boolean is_active
    timestamp created_at
}

categories {
    int id PK
    varchar name
    varchar slug
    int parent_id FK
}
```

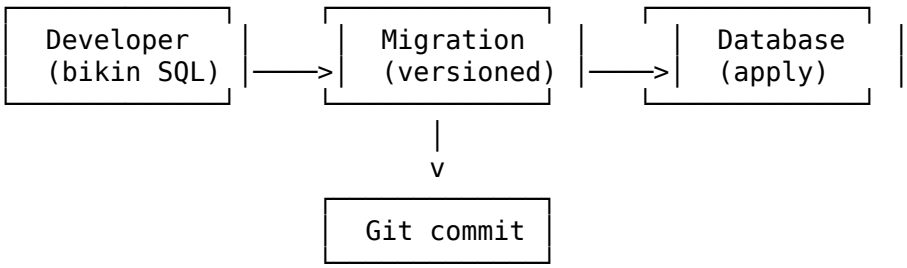
```
tags {
    int id PK
    varchar name
    varchar slug
}

product_tags {
    int product_id PK,FK
    int tag_id PK,FK
}
```

Cara Baca ERD

1. Kotak = entity / table
2. Baris di dalam kotak = column
3. PK = Primary Key, FK = Foreign Key
4. Garis = relationship
5. Simbol garis = cardinality

Schema Migration Workflow

Workflow development database yang proper:



Langkah-langkah

1. **Tulis migration** — file SQL dengan nomor urut
2. **Review** — di code review bareng tim
3. **Apply ke dev** — jalanin migration di database development
4. **Test** — pastikan aplikasi masih jalan
5. **Commit** — masuk ke git
6. **Deploy** — apply ke staging, lalu production

Contoh Workflow

```
# Struktur folder migrations
migrations/
├── 001_create_users.sql
├── 002_create_products.sql
├── 003_create_orders.sql
└── 004_add_category_to_products.sql
```

```sql
-- 004_add_category_to_products.sql
-- Up
ALTER TABLE products ADD COLUMN category_id INT REFERENCES categories(id);
CREATE INDEX idx_products_category ON products(category_id);

-- Down
DROP INDEX IF EXISTS idx_products_category;
ALTER TABLE products DROP COLUMN IF EXISTS category_id;
```

Migration Best Practices

- **Jangan edit migration yang sudah di-commit** — bikin migration baru aja
- **Setiap migration harus punya rollback (Down)** — biar bisa revert
- **Test migration di dev/staging dulu** — sebelum production
- **One migration = one logical change** — jangan campur-campur
- **Jangan manual edit database production** — selalu lewat migration

---

Database vs Application-Level Validation

Validasi data bisa di dua tempat: database (constraint) atau aplikasi (code).

Database Validation

```sql
CREATE TABLE users (
    id SERIAL PRIMARY KEY,
    email VARCHAR(255) UNIQUE NOT NULL,              -- unique + not null
    age INT CHECK (age >= 17 AND age <= 150),        -- check constraint
    name VARCHAR(100) NOT NULL,
    price DECIMAL(10, 2) CHECK (price > 0),
    status VARCHAR(20) DEFAULT 'active'
```

```
        CHECK (status IN ('active', 'inactive', 'banned')),
    created_at TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);
```

Application Validation

```
// Express.js — validation di aplikasi
const Joi = require('joi');

const userSchema = Joi.object({
    email: Joi.string().email().required(),
    age: Joi.number().min(17).max(150).required(),
    name: Joi.string().min(2).max(100).required(),
    price: Joi.number().positive().precision(2),
    status: Joi.string().valid('active', 'inactive', 'banned')
});
```

Perbandingan

Aspek	Database	Aplikasi
Kecepatan	Lambat (I/O)	Cepat (in-memory)
Keamanan	□ Last defense — bypass aplikasi tetap kena	□ Bisa di-skip
Pesan error	□ Generic / cryptic	□ Bisa custom
Data consistency	□ Semua entry point kena	□ Tiap aplikasi beda
Business logic	□ Sulit (complex rules)	□ Mudah

> **Best practice:** DOUBLE VALIDATION. Validasi di aplikasi buat UX cepat + error message bagus. Validasi di database sebagai **safety net** — tetap pake NOT NULL, UNIQUE, CHECK, FK.

Latihan

Latihan 1: Normalisasi Table

Table berikut dalam bentuk unnormalized. Normalisasi sampai 3NF:

Sales Table:

SalesID	Date	Customer	Phone	Product1	Qty1	Price1	Product2	Qty2	Price2	Total
1	2024-01-10	Budi	0812...	Kopi	2	45000	Teh	1	25000	115000
2	2024-01-10	Ani	0821...	Gula	3	15500	NULL	NULL	NULL	46500
3	2024-01-11	Budi	0812...	Kopi	1	45000	NULL	NULL	NULL	45000

Identifikasi masalahnya, lalu buat struktur table 3NF.

Latihan 2: Desain E-commerce Schema

Buat desain database lengkap untuk aplikasi toko online dengan fitur: - User bisa register & login - User bisa lihat product by category - User bisa add to cart - User bisa checkout (cart → order) - Admin bisa manage product & category - Setiap product bisa punya banyak gambar - Track order status (pending → paid → shipped → delivered)

Tentukan: 1. Semua table yang diperlukan 2. Column dan tipe data masing-masing 3. Primary key & foreign key 4. Index yang diperlukan 5. Constraint (UNIQUE, NOT NULL, CHECK)

Latihan 3: Tulis Migration

Buat 3 file migration berurutan untuk aplikasi blog:

- `001_create_users.sql` — users table
- `002_create_posts.sql` — posts table (dengan FK ke users)
- `003_add_published_at.sql` — tambah column published_at ke posts

Tiap migration harus punya bagian UP dan DOWN.

Latihan 4: ERD dari Requirement

Gambarkan ERD (dalam format mermaid) untuk sistem perpustakaan dengan requirement:

- Anggota bisa pinjam buku
- Satu anggota bisa pinjam max 3 buku
- Buku punya kategori
- Satu buku bisa punya banyak penulis
- Tracking peminjaman: tgl pinjam, tgl harus kembali, tgl kembali (null if belum)
- Denda keterlambatan: Rp 1000/hari